Лабораторна робота 11. JavaScript та життєвий цикл компонентів Blazor

Завдання та очікуваний результат

Додайте можливості браузера до наявного застосунку: підтвердження видалення, копіювання підсумку та зворотне повідомлення від JavaScript до C#. Покажіть зміну навчального статусу за часом і коректне завершення фонової роботи.

1. Власний JS-модуль, виклики C# → JS і JS → C#.
2. Скасування видалення залишає позицію в кошику.
3. Копіювання має зрозумілий результат і запасний ручний варіант.
4. Таймер не працює після виходу зі сторінки.
5. Розуміння `OnInitialized`, `OnParametersSetAsync`, `OnAfterRenderAsync` та `DisposeAsync`.
6. Додатковий рівень: узагальнений компонент пагінації. У готовому демо цей рівень також реалізовано.

Продовжуйте власний проєкт із лабораторної 10. Не створюйте новий шаблон і не замінюйте свою тему демонстраційною. Перед початком збережіть попередній результат комітом або окремою копією.

Готовий контрольний варіант — папка `Lab11_CampusCafe`. У ній застосунок `CampusCafe`, перевірки `CampusCafe.Checks` та README. Демо є результатом саме цього етапу, а не початковим шаблоном для механічної заміни власного проєкту.

Межі навчального застосунку: оплати, автентифікації та розмежування доступу немає. Список замовлень спільний, а не захищений список «моїх» покупок. Запускайте демо локально та вводьте тільки вигадані дані. Публікація вихідного коду не означає готовність сервісу приймати реальних покупців.

Як працювати з кодом

У прикладі збережено назви `CampusCafe`, `Drink` і кавові опції. У своєму застосунку продовжуйте тему, обрану раніше: букети, фотодрук, подарункові набори, сендвічі чи інший каталог. Під час замін не втрачайте власні назви, формули та оформлення.

Повні лістинги показують узгоджений результат кроку. Перед додаванням коду визначте місце його виконання: сервер C#, браузер JavaScript чи розмітка компонента. Для невеликих правок використано `diff`: рядок із `-` вилучається, із `+` додається без знака `+`; рядки без цих знаків — контекст. Заголовки `---`, `+++` і `@@` не копіюйте до програми. Для великих змін наведено повні версії «Було / Стало», для CSS — лише новий файл.

Для кожної асинхронної операції дайте відповіді: хто її запускає, який результат очікується, що станеться при відмові та хто звільнить ресурси після виходу зі сторінки. Перевіряйте і успішні, і скасовані дії. Назви кнопок та етапів адаптуйте до власної теми, зберігаючи зміст механізмів.

Шляхи в лістингах відраховано від папки `CampusCafe`, де лежить `.csproj`. Це не вимога називати власний проєкт так само. Після зміни простору імен узгоджуйте його в усіх файлах через Rename редактора.

Після завершення кожного логічного етапу виконуйте `dotnet build`. Нові взаємозалежні класи спочатку додайте всі, а потім збирайте. Контрольні точки нижче вказують, коли очікується повністю працездатна версія.

1. Де виконується код

C# компонентів цього проєкту виконується на сервері. Буфер обміну й стандартний діалог `confirm` належать браузеру. `IJSRuntime` передає виклик браузеру і повертає результат асинхронно.

```
Натискання → C# → IJSRuntime → JS-модуль → браузерний API
                                    ↓
                        invokeMethodAsync → C# → повідомлення
```

Не переносимо весь інтерфейс у JavaScript і не змінюємо DOM, яким керує Blazor. Окремий ES-модуль не засмічує глобальний `window` власними функціями. У `App.razor` не потрібно додавати другий скрипт Blazor або посилання на `_Host.cshtml`.

Перед початком повторіть `async/await`: очікування JS не означає, що весь сервер зупинено. Проте код після `await` має бути готовим до того, що користувач уже залишив сторінку.

2. JavaScript-модуль

Створіть папку `wwwroot/js`. `export` робить функцію доступною після `import`. `confirmRemoval` повертає `bool` у C#. `copySummary` повертає `false`, якщо браузер не надає Clipboard API або відхилив доступ.

Копіювання викликається користувацьким натисканням, але політики браузера можуть його заблокувати, зокрема через затримку серверного виклику. Для Clipboard API потрібний безпечний контекст: HTTPS або дозволений браузером localhost. HTTP-адреса іншого комп'ютера в локальній мережі не рівнозначна localhost.

Після успіху `invokeMethodAsync("OnCopied")` викликає метод конкретного C#-компонента. Це окремий навчальний приклад зворотного напрямку; у простішій реалізації можна було б обмежитися поверненням `bool`.

Файл `wwwroot/js/orderTools.js`

Розбір першої функції. `confirmRemoval(name)` отримує назву товару від C#. Шаблонний рядок із `${name}` формує запитання, а `window.confirm` показує стандартний діалог браузера. Повернене `true` означає підтвердження, `false` — відмову. JavaScript тут не видаляє товар: рішення про зміну кошика залишається в C#.

Розбір другої функції. `copySummary(text, receiver)` отримує текст і посилання для зворотного виклику. Спочатку вона перевіряє доступність браузерного API. `writeText` повертає Promise, тому `await` очікує завершення операції. Наявність `navigator.clipboard` не гарантує дозволу, через що потрібен і `catch`.

Після підтвердженого запису функція звертається до `receiver.invokeMethodAsync("OnCopied")`. Це виклик методу конкретного .NET-об'єкта, а не пошук довільної функції з таким ім'ям у проєкті. Назва повинна збігатися з методом компонента, доступним через `[JSInvokable]`.

Зворотний виклик стоїть поза `catch` операції копіювання. Тому збій callback може статися вже після запису тексту в буфер. Надалі компонент показує для такої невизначеної ситуації загальну помилку й ручний варіант. `false` у цьому модулі означає відмову саме на етапі доступу до Clipboard API.

Прогноз перед запуском: які дії завершаться без виклику `OnCopied`? Знайдіть усі такі гілки. Не намагайтеся обійти заборону браузера: застосунок має коректно запропонувати альтернативу.

Створіть модуль із двома функціями:

```
export function confirmRemoval(name) {
    return window.confirm(`Видалити «${name}» з кошика?`);
}

export async function copySummary(text, receiver) {
    if (!window.isSecureContext || !navigator.clipboard) return false;
    try {
        await navigator.clipboard.writeText(text);
    } catch {
        return false;
    }
    // Приклад зворотного виклику JavaScript → екземпляр компонента C#.
    await receiver.invokeMethodAsync("OnCopied");
    return true;
}
```

3. Підтвердження видалення

`InvokeAsync<IJSObjectReference>("import", ...)` завантажує модуль і повертає посилання на нього. Потім викликаємо його функцію за ім'ям. Для цієї короткої операції посилання звільняється через `await using`; браузер кешує сам модуль.

Якщо користувач натиснув «Скасувати», `Order.Remove` не викликається. Помилка JS не повинна призводити до видалення. Втрата circuit означає, що взаємодія з браузером уже недоступна.

Контроль: додайте дві позиції, скасуйте видалення першої, підтвердіть видалення другої. Сума має змінитися лише у другому випадку.

Файл `Components/Shared/CartSummary.razor`

Що змінює diff. Раніше `@onclick` одразу викликав `Order.Remove(item.Id)`. Тепер натискання викликає `ConfirmRemoveAsync(item)`, а видалення залишається лише всередині гілки позитивної відповіді. Лямбда `() => ...` передає саме ту позицію, біля якої натиснули кнопку.

`@inject IJSRuntime JS` надає інструмент зв'язку з браузером. Перший `InvokeAsync` виконує `import` і повертає `IJSObjectReference`: через цей об'єкт C# звертається до експортованих функцій модуля. У другому виклику `<bool>` указує очікуваний тип результату. JS-значення `true` / `false` перетворюється на C# `bool`.

`await using` звільняє .NET-посилання на JS-об'єкт після завершення операції. Це не видаляє сам файл із браузерного кешу. `JSException` означає помилку виконання JavaScript, а `JSDisconnectedException` — недоступність зв'язку з браузером. В обох випадках команда видалення не повинна виконуватися замість підтвердження.

Наявна підписка `Order.Changed += Refresh` залишається: після успішного `Remove` стан кошика повідомляє про зміну, і компонент оновлює суму. Підписку прибирає звичайний `Dispose`, тоді як локальне посилання на модуль звільняє `await using`.

Контрольна точка: завершивши модуль і цей diff, збережіть застосунок і перевірте відмову та підтвердження. Перед натисканням запишіть очікувану суму кошика після кожної дії.

Внесіть локальні зміни:

```
--- a/Components/Shared/CartSummary.razor
+++ b/Components/Shared/CartSummary.razor
@@ -1,5 +1,6 @@
 @implements IDisposable
 @inject OrderState Order
+@inject IJSRuntime JS

 <aside class="cart" aria-label="Кошик">
     <span class="eyebrow">Ваша перерва</span>
@@ -18,7 +19,7 @@
                     <p>@item.Volume мл · @item.Milk молоко · @item.Quantity шт.</p>
                     <div class="cart-line-footer">
                         <MoneyLabel Amount="item.Total" />
-                        <button type="button" class="text-button" @onclick="() =>
Order.Remove(item.Id)"
+                        <button type="button" class="text-button" @onclick="() =>
ConfirmRemoveAsync(item)"
                                 aria-label="@($"Видалити
{item.DrinkName}")">Видалити</button>
                     </div>
                 </li>
@@ -31,10 +32,24 @@
         <a class="button-link checkout-link" href="/checkout">Перевірити замовлення</a>
     }
     <p class="cart-note">Кошик тимчасовий. Замовлення зберігається після підтвердження на
окремій сторінці. Оплати немає.</p>
+    @if (jsError is not null) { <p role="alert">@jsError</p> }
 </aside>
```

```
 @code {
+    private string? jsError;
     protected override void OnInitialized() => Order.Changed += Refresh;
     private void Refresh() => _ = InvokeAsync(StateHasChanged);
     public void Dispose() => Order.Changed -= Refresh;
+    private async Task ConfirmRemoveAsync(OrderLine item)
+    {
+        jsError = null;
+        try
+        {
+            await using var module = await JS.InvokeAsync<IJSObjectReference>("import",
"./js/orderTools.js");
+            if (await module.InvokeAsync<bool>("confirmRemoval", item.DrinkName))
+                Order.Remove(item.Id);
+        }
+        catch (JSDisconnectedException) { }
+        catch (JSException) { jsError = "Не вдалося відкрити підтвердження. Позицію не
видалено."; }
+    }
 }
```

4. Копіювання й зворотний виклик

Компонент імпортує модуль в `OnAfterRenderAsync` лише за `firstRender`. До першого інтерактивного рендерингу браузерний інтероп не слід використовувати. Після асинхронного імпорту викликаємо `StateHasChanged`, щоб розблокувати кнопку; завершення `OnAfterRenderAsync` саме по собі не запускає наступний рендер.

`DotNetObjectReference.Create(this)` надає JS посилання на цей екземпляр компонента. `[JSInvokable]` дозволяє викликати `OnCopied`, який оновлює повідомлення в контексті рендерера. У `DisposeAsync` звільняємо і JS-посилання, і посилання на .NET-об’єкт.

Підсумок не містить телефону й адреси. За відмови браузера показується текст для ручного копіювання. Не можна повідомляти «скопійовано», доки браузер не підтвердив операцію.

Адаптуйте зараз: короткий підсумок свого замовлення. Не включайте персональні дані до тексту без потреби.

Файл `Components/Shared/OrderTools.razor`

Які дані зберігає компонент. `Summary` отримує текст від батьківської сторінки. `module` представляє JS-модуль, а `receiver` — поточний .NET-компонент, до якого звертатиметься JavaScript. Це два різні напрямки зв’язку. `busy` блокує повторну операцію, `showFallback` вмикає ручний варіант, `message` зберігає повідомлення, `disposed` позначає завершення життя компонента.

Прочитайте методи в порядку роботи користувача:

1. **Перше відображення.** Кнопка заблокована, оскільки `module` ще `null`. В `OnAfterRenderAsync` завантажується модуль і створюється `receiver`. Перевірка `firstRender` не допускає

повторного імпорту після кожного оновлення тексту.

2. **Завершення завантаження.** `StateHasChanged()` запитує ще один рендеринг, щоб кнопка врахувала появу модуля. Тут це потрібно явно: завершення `OnAfterRenderAsync` саме такого оновлення не гарантує.
3. **Натискання.** `CopyAsync` перевіряє готовність посилань і стан `busy`, передає `Summary` та `receiver` до JS. Якщо JS повертає `false`, показується текстова область для ручного копіювання.
4. **Успішний callback.** JavaScript викликає `OnCopied`. Метод змінює повідомлення та запитує відображення через `InvokeAsync`, тобто в контексті рендерера Blazor.
5. **Вихід зі сторінки.** `DisposeAsync` позначає компонент завершеним і звільняє обидва посилання.

Навіщо перевірка `disposed` після `await`. Поки браузер імпортує модуль, користувач може перейти на іншу сторінку. Повернений пізніше об’єкт не можна просто залишити в уже видаленому компоненті: код одразу звільняє його. Аналогічно callback не повинен оновлювати повідомлення завершеного компонента.

`receiver?.Dispose()` дозволяє прибрати службове посилання на .NET-об’єкт. `module.DisposeAsync()` звільняє посилання на JS-об’єкт. Це не рівнозначне автоматичному скасуванню всіх можливих JS-таймерів: якби модуль створював такі ресурси, для них знадобилося б окреме завершення.

Розмітка запасного варіанта. `readonly` дозволяє виділити текст, але не редагувати його в цьому полі. `role="status"` позначає повідомлення про результат. Сам текст підсумку не повинен включати адресу чи телефон лише тому, що вони доступні в замовленні.

Перевірте розуміння: чому після успішного виклику `writeText` саме `OnCopied` установлює текст успіху? Для простішого варіанта можна було б використати лише повернене `bool`; тут callback додано для вивчення напрямку JS → C#.

Створіть компонент, простеживши життєвий цикл обох посилань:

```
@implements IAsyncDisposable
@inject IJSRuntime JS

<div class="order-tools">
    <button type="button" disabled="@(module is null || busy)"
@onclick="CopyAsync">Копіювати підсумок</button>
    <p role="status">@message</p>
    @if (showFallback)
    {
        <label for="copy-fallback">Скопіюйте текст вручну</label>
        <textarea id="copy-fallback" readonly rows="4" value="@Summary"></textarea>
    }
</div>

@code {
    [Parameter, EditorRequired] public string Summary { get; set; } = "";
    private IJSObjectReference? module;
    private DotNetObjectReference<OrderTools>? receiver;
    private bool busy;
    private bool disposed;
```

```
    private bool showFallback;
    private string? message;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (!firstRender) return;
        try
        {
            var imported = await JS.InvokeAsync<IJSObjectReference>("import",
"./js/orderTools.js");
            if (disposed) { await imported.DisposeAsync(); return; }
            module = imported;
            receiver = DotNetObjectReference.Create(this);
        }
        catch (JSException) { message = "Модуль не завантажено. Скористайтеся текстовим
підсумком."; showFallback = true; }
        catch (JSDisconnectedException) { return; }
        if (!disposed) StateHasChanged();
    }

    private async Task CopyAsync()
    {
        if (module is null || receiver is null || busy) return;
        busy = true;
        showFallback = false;
        try
        {
            if (!await module.InvokeAsync<bool>("copySummary", Summary, receiver))
            {
                message = "Браузер не дозволив копіювання. Виділіть текст нижче.";
                showFallback = true;
            }
        }
        catch (JSException) { message = "Помилка JavaScript. Скопіюйте текст вручну.";
showFallback = true; }
        catch (JSDisconnectedException) { }
        finally { busy = false; }
    }

    [JSInvokable]
    public Task OnCopied() => InvokeAsync(() =>
    {
        if (disposed) return;
        message = "Підсумок скопійовано.";
        StateHasChanged();
    });

    public async ValueTask DisposeAsync()
    {
        disposed = true;
        receiver?.Dispose();
        if (module is not null)
        {
            try { await module.DisposeAsync(); }
            catch (JSDisconnectedException) { }
        }
    }
}
```

5. Статус і фоновий таймер

Це явно позначена імітація, а не стеження за реальним замовленням: статус обчислюється за різницею між поточним UTC-часом і `CreatedUtc`. За 15 секунд він стає «Готується», за 30 — «Готове до отримання». Дані статусу не записуються до БД, а рядки БД не опитуються кожні три секунди.

Відокремлення формули статусу дозволяє перевірити часові межі без очікування таймера. `PeriodicTimer` використано замість нескінченного циклу без затримки. `CancellationTokenSource` завершує очікування під час виходу зі сторінки. `DisposeAsync` очікує завершення задачі й звільняє ресурс.

`StateHasChanged` потрібний тут, бо зміна надійшла від таймера, а не звичайної UI-події. `InvokeAsync` виконує оновлення через контекст рендерера. Для іншого замовлення батьківський компонент використає новий `@key`, отже старий таймер буде завершено.

Контроль: відкрийте нове замовлення на 35 секунд; статус змінюється. Потім поверніться до списку й відкрийте інше. Помилок завершеного компонента бути не повинно.

Файл `Models/DemoOrderProgress.cs`

Чиста функція для навчального статусу. Клас не звертається до БД і не знає про Razor. Метод отримує два моменти часу, віднімає їх та вибирає результат через `switch`. Передавання `nowUtc` параметром дозволяє тесту підставити будь-який момент без реального очікування.

При різниці 14 секунд повертається «Прийнято», при 15 — «Готується», при 30 — «Готове до отримання». `IsComplete` визначає, коли періодичне оновлення вже не потрібне. Якщо адаптуєте межі або назви, узгодьте обидва методи, пояснення в інтерфейсі та тести.

Контроль без таймера: для одного `createdUtc` розгляньте `createdUtc.AddSeconds(14)`, `AddSeconds(15)`, `AddSeconds(29)` і `AddSeconds(30)`. Запишіть очікуваний текст та результат `IsComplete`, щоб перевірити межі алгоритму.

Створіть окремий клас формули:

```
namespace CampusCafe.Models;

public static class DemoOrderProgress
{
    public static string GetStatus(DateTime createdUtc, DateTime nowUtc)
    {
        var age = nowUtc - createdUtc;
        return age.TotalSeconds switch
        {
            < 15 => "Прийнято",
            < 30 => "Готується",
            _ => "Готове до отримання"
        };
    }
```

```
    public static bool IsComplete(DateTime createdUtc, DateTime nowUtc)
        => (nowUtc - createdUtc).TotalSeconds >= 30;
}
```

Файл `Components/Shared/LiveOrderStatus.razor`

Хто за що відповідає. Формула в `DemoOrderProgress` визначає статус, а цей компонент вирішує, коли його перерахувати та показати. `CreatedUtc` приходить із батьківської сторінки. Поле `status` містить поточний текст, `loop` зберігає задачу фонового циклу, `stop` дає можливість попросити її завершитися.

`OnInitialized` одразу викликає `UpdateStatus`, тому старе замовлення відразу показує відповідний результат. Далі `loop = RunAsync()` запускає асинхронну операцію та зберігає її `Task`. Це не створення окремого потоку через `Task.Run` і не безперервне зайняття процесора.

У `RunAsync` цикл очікує `WaitForNextTickAsync`, а не виконується без паузи. Після сигналу таймера `InvokeAsync` передає зміну стану в контекст Blazor. `StateHasChanged()` потрібний, бо джерелом зміни є таймер, а не звичайне натискання користувача.

Як припиняється робота. `stop.Cancel()` подає сигнал скасування. Очікування з цим токеном може завершитися `OperationCanceledException`, яку код розпізнає як нормальне завершення. `await loop` дочікується виходу з циклу, а `using var timer` звільняє сам таймер. Потім звільняється джерело скасування.

Коли замовлення досягло завершального статусу, наступне виконання умови припиняє цикл. Межа статусу визначається формулою, а момент її появи на екрані залежить від періоду оновлення і планування виконання. Цей компонент не підтверджує роботу кухні й не отримує нових даних від персоналу.

Перевірте розуміння: чому перехід назад до старого замовлення не повинен починати його приготування заново? Відповідь міститься в тому, який момент передано через `CreatedUtc`. Для справжнього відстеження джерелом статусу був би серверний запис або повідомлення, а не сама різниця часу.

Створіть компонент оновлення статусу:

```
@implements IAsyncDisposable

<div class="status-panel" role="status">
    <strong>@status</strong>
    <p>Навчальна імітація: 0–15 с — прийнято; 15–30 с — готується; далі — готове.</p>
    <small>Це не реальне відстеження кухні чи доставки. Перевірка кожні 3 секунди.</small>
</div>

@code {
    [Parameter] public DateTime CreatedUtc { get; set; }
    private readonly CancellationTokenSource stop = new();
    private Task? loop;
    private string status = "";

    protected override void OnInitialized()
```

```
    {
        UpdateStatus();
        loop = RunAsync();
    }
    private void UpdateStatus() => status = DemoOrderProgress.GetStatus(CreatedUtc,
DateTime.UtcNow);

    private async Task RunAsync()
    {
        using var timer = new PeriodicTimer(TimeSpan.FromSeconds(3));
        try
        {
            while (!DemoOrderProgress.IsComplete(CreatedUtc, DateTime.UtcNow)
                && await timer.WaitForNextTickAsync(stop.Token))
            {
                await InvokeAsync(() => { UpdateStatus(); StateHasChanged(); });
            }
        }
        catch (OperationCanceledException) when (stop.IsCancellationRequested) { }
    }

    public async ValueTask DisposeAsync()
    {
        stop.Cancel();
        if (loop is not null) await loop;
        stop.Dispose();
    }
}
```

6. Підключення до деталей замовлення

Додаємо два компоненти після підсумку. Вони отримують лише потрібні дані: час створення та безпечний текст. `@key="order.Id"` змушує Blazor створити інші екземпляри при переході до іншого номера.

Після цього основний функціональний етап завершено. Пагінація нижче — додатковий рівень; її відсутність не перешкоджає JS-вправам і таймеру.

Файл `Components/Pages/OrderDetails.razor`

Складання готових частин. Сторінка вже завантажила `order` із БД, тому новим компонентам не потрібно повторно читати замовлення. `LiveOrderStatus` отримує час, `OrderTools` — сформований рядок. Це приклад вузьких параметрів: кожна частина отримує лише необхідне.

`@key` задає ідентичність компонента серед сусідів. При зміні номера замовлення старий екземпляр замінюється новим, що запускає звільнення старих ресурсів. Це важливо для `LiveOrderStatus`, де запуск відбувається в `OnInitialized`, а не при кожній зміні параметра. На цій сторінці компоненти також прибираються на етапі завантаження нового запису.

Адаптація: замініть назву закладу та зміст підсумку, але залиште номер і зрозумілий результат. Перевірте, що сума у скопійованому тексті відповідає збереженому замовленню, а не поточному кошику.

Внесіть локальні зміни:

```
--- a/Components/Pages/OrderDetails.razor
+++ b/Components/Pages/OrderDetails.razor
@@ -23,6 +23,8 @@
         </ul>
         <p class="review-total">Разом: <MoneyLabel Amount="order.Total" /></p>
         <p>Замовлення збережено. Оплата не проводилася.</p>
+        <LiveOrderStatus @key="order.Id" CreatedUtc="order.CreatedUtc" />
+        <OrderTools @key="order.Id" Summary="@($"Campus Café — замовлення №{order.Id},
{order.Items.Sum(i => i.Quantity)} шт., {order.Total:N2} грн. Оплати не було.")" />
         @if (!string.IsNullOrEmpty(order.CustomerName))
         {
             <h2>Отримання</h2>
```

7. Оформлення нових компонентів

Замініть тільки `orders.css` наведеною повною версією. Стилі `.pager` знадобляться для додаткової вправи, решта — для статусу й копіювання.

Файл `wwwroot/orders.css`

Роль нових селекторів. `.status-panel` відділяє навчальний статус від інших даних. `.order-tools textarea` робить ручний підсумок доступним по ширині блока. `.pager` вирівнює кнопки та номер сторінки. `var(--green)` читає CSS-змінну попереднього оформлення: змінюючи тему, перевірте, що ця змінна лишилася визначеною або має ваш відповідник.

Стилі форм і списку з лабораторних 9–10 залишено, щоб нова версія не зламала попередні сторінки. CSS не запускає таймер, не надає доступу до буфера і не змінює статус у БД. Після заміни перевірте також оформлення замовлення та вузький екран.

Замініть вміст файла. Нижче наведено лише нову повну версію; власну палітру з `app.css` збережіть.

```
.panel { background:#fff; border-radius:8px; padding:24px; margin:20px 0; box-shadow:0 2px
7px #0002; }
.review-lines { list-style:none; padding:0; }
.review-lines li { display:flex; flex-wrap:wrap; justify-content:space-between; gap:12px;
padding:14px 0; border-bottom:1px solid #ddd; }
.review-lines li span { color:#555; }
.review-total { font-size:1.3rem; font-weight:700; }
.orders-list { max-width:850px; }
.checkout-link { display:block; margin-top:14px; text-align:center; }
.back-link { display:inline-block; margin:18px; }
button:disabled { opacity:.55; cursor:not-allowed; }
nav { flex-wrap:wrap; }
.status-panel { border-left:4px solid var(--green); padding:12px 18px; background:#f2f8ef;
margin:20px 0; }
.order-tools { margin-top:20px; }
.order-tools textarea { display:block; width:100%; font:inherit; }
.pager { display:flex; gap:16px; align-items:center; margin:22px 0; }
```

```
.checkout-grid { display:grid; grid-template-columns:1fr 1fr; gap:24px; }
.form-fields { display:grid; gap:8px; }
.form-fields label { font-weight:700; margin-top:12px; }
.form-fields input,.form-fields textarea { width:100%; padding:10px; border:1px solid #999;
border-radius:4px; font:inherit; }
fieldset { border:0; padding:0; margin:0; min-width:0; }
.form-hint { color:#555; font-size:.9rem; }
@media (max-width:760px) { .checkout-grid { grid-template-columns:1fr; } }
```

8. Додатковий рівень — шаблонна пагінація

Мета — зробити компонент, який знає, як ділити список на сторінки, але не знає, як виглядає замовлення. Це окрема додаткова вправа на повторне використання компонента, а не необхідна умова роботи JavaScript чи таймера.

`@typeparam TItem` оголошує тип елемента. `RenderFragment<TItem>` — шаблон розмітки, який отримує один елемент цього типу. Батьківська сторінка визначає вигляд картки через `ItemTemplate`, а `Context="order"` задає ім'я її змінної.

Це навчальна пагінація вже завантаженого списку. Для тисяч замовлень потрібно застосовувати `Skip` / `Take` у запиті БД, а не завантажувати все.

Компонент перевіряє `PageSize`, обробляє порожню колекцію та коригує номер сторінки, якщо список скоротився. Кнопки справді є `<button disabled>`, а не `<li disabled>`. Логіка методів також обмежує межі.

Після створення компонента змініть лише відображення списку. Завантаження й сервіс залишаються тими самими. Створіть щонайменше чотири замовлення, щоб перевірити другу сторінку.

Файл `Components/Shared/PagedList.razor`

Розбір узагальнення. `TItem` — місце для типу елемента, який визначить батьківський компонент. Для замовлень це `PurchaseOrder`, для каталогу міг би бути `Drink`. `Items` містить дані, а `ItemTemplate` описує спосіб відображення одного елемента. Виклик `@ItemTemplate(item)` передає поточний об'єкт до цього шаблону.

Розбір обчислень. Усередині `page` починається з нуля. Для `PageSize = 3` перша сторінка виконує `Skip(0).Take(3)`, друга — `Skip(3).Take(3)`. Користувачу показуємо `page + 1`. Ділення на `(double)PageSize` зберігає дробову частину, а `Math.Ceiling` округлює вгору: чотири елементи потребують двох сторінок, а не однієї.

`OnParametersSet` перевіряє додатний розмір сторінки та обмежує поточний індекс через `Math.Clamp`. Наприклад, якщо користувач був на другій сторінці, а список скоротився до двох елементів, індекс має повернутися до першої. Гілка `Items.Count == 0` показує `EmptyContent`, не намагаючись відобразити картки.

Кнопки обмежено двічі: атрибут `disabled` робить межу видимою й недоступною в UI, а `Math.Max` / `Math.Min` у методах захищають сам стан. При повторному використанні компонента

логіка не залежить від того, як намальовано кнопки.

Контроль: вручну обчисліть `PageCount` і діапазон елементів для 0, 1, 3, 4 та 7 записів. Потім перевірте відповідні випадки в браузері. Для порожнього списку поточна батьківська сторінка вже має власне повідомлення; `EmptyContent` потрібний для інших варіантів використання компонента.

Створіть компонент пагінації:

```
@typeparam TItem

@if (Items.Count == 0)
{
    @EmptyContent
}
else
{
    @foreach (var item in Items.Skip(page * PageSize).Take(PageSize))
    {
        @ItemTemplate(item)
    }
    <nav class="pager" aria-label="Сторінки списку">
        <button type="button" disabled="@(page == 0)" @onclick="Previous">Назад</button>
        <span>Сторінка @(page + 1) із @PageCount</span>
        <button type="button" disabled="@(page >= PageCount - 1)"
@onclick="Next">Далі</button>
    </nav>
}

@code {
    [Parameter, EditorRequired] public IReadOnlyList<TItem> Items { get; set; } =
Array.Empty<TItem>();
    [Parameter, EditorRequired] public RenderFragment<TItem> ItemTemplate { get; set; } =
default!;
    [Parameter] public RenderFragment? EmptyContent { get; set; }
    [Parameter] public int PageSize { get; set; } = 3;
    private int page;
    private int PageCount => Math.Max(1, (int)Math.Ceiling(Items.Count /
(double)PageSize));
    protected override void OnParametersSet()
    {
        if (PageSize <= 0) throw new ArgumentOutOfRangeException(nameof(PageSize));
        page = Math.Clamp(page, 0, PageCount - 1);
    }
    private void Previous() => page = Math.Max(0, page - 1);
    private void Next() => page = Math.Min(PageCount - 1, page + 1);
}
```

Файл `Components/Pages/Orders.razor`

Що переноситься в дочірній компонент. Зникає лише зовнішній `foreach`. `PagedList` тепер вибирає, які елементи показувати, а розмітка `article` залишається шаблоном у батьківській сторінці. `Context="order"` задає локальну назву об’єкта всередині шаблону, а не виконує запит до БД.

`TItem="PurchaseOrder"` повідомляє тип, `Items="orders"` передає вже завантажений список, `PageSize="3"` задає кількість карток. Метод завантаження, повідомлення про помилки та посилання на деталі не переписуються. Отже, додавання пагінації змінює відображення, а не спосіб збереження даних.

Самостійний перенос: для другого типу даних спочатку визначте `TItem`, джерело `Items` і поля картки. Замініть лише шаблон та параметри. Якщо довелося дописувати всередині `PagedList` умову для кави або замовлень, відповідальність компонента змішалася з тематикою сторінки.

Внесіть локальні зміни:

```
--- a/Components/Pages/Orders.razor
+++ b/Components/Pages/Orders.razor
@@ -15,14 +15,15 @@
 else
 {
     <div class="orders-list">
-        @foreach (var order in orders)
-        {
+        <PagedList TItem="PurchaseOrder" Items="orders" PageSize="3">
+            <ItemTemplate Context="order">
             <article class="panel" @key="order.Id">
                 <h2><a href="@($"/orders/{order.Id}")">Замовлення №@order.Id</a></h2>
                 <p>@order.CreatedUtc.ToString("dd.MM.yyyy HH:mm") UTC · @order.Items.Sum(i
=> i.Quantity) шт.</p>
                 <MoneyLabel Amount="order.Total" />
             </article>
-        }
+            </ItemTemplate>
+        </PagedList>
     </div>
 }
```

9. Перевірки та самостійна частина

Дія	Очікуваний результат
Скасувати JS-підтвердження видалення	Позиція залишається
Підтвердити видалення	Позиція зникає, сума оновлюється
Копіювати з дозволеним Clipboard API	Повідомлення успіху, правильний текст у буфері
Браузер заборонив Clipboard API	Повідомлення й ручний варіант, без хибного успіху
Залишитися на новому замовленні	Протягом 30–33 с статус доходить до готовності
Вийти зі сторінки раніше	Очікування таймера скасоване
Повернутися до старого замовлення	Статус відповідає його часу, а не часу відкриття сторінки
Пагінація: 0, 1, 3, 4 записи	Немає помилок меж, правильна кількість сторінок

Самостійно: адаптуйте текст JS-підтвердження, підсумок і назви етапів під свою тематику. Поясніть, чому автоматичний часовий статус не можна видавати за справжнє підтвердження виконання.

Для додаткового рівня використайте `PagedList` із другим типом даних, наприклад товарами: шаблон зміниться, але логіку пагінації переписувати не потрібно. Зовнішню бібліотеку діалогів можна дослідити окремо; не використовуйте залежність `@latest` у відтворюваному навчальному проєкті.

Структура результату

```
CampusCafe/
  Components/App.razor
  Components/Layout/MainLayout.razor
  Components/Layout/MainLayout.razor.css
  Components/Layout/ReconnectModal.razor
  Components/Layout/ReconnectModal.razor.css
  Components/Layout/ReconnectModal.razor.js
  Components/Pages/About.razor
  Components/Pages/Checkout.razor
  Components/Pages/Error.razor
  Components/Pages/Home.razor
  Components/Pages/NotFound.razor
  Components/Pages/OrderDetails.razor
  Components/Pages/Orders.razor
  Components/Routes.razor
  Components/Shared/CartSummary.razor
  Components/Shared/ConfigureDrinkDialog.razor
  Components/Shared/CustomerFields.razor
  Components/Shared/DrinkCard.razor
```

```
Components/Shared/LiveOrderStatus.razor
Components/Shared/MoneyLabel.razor
Components/Shared/OrderReview.razor
Components/Shared/OrderTools.razor
Components/Shared/PagedList.razor
Components/_Imports.razor
Data/CafeDbContext.cs
Data/OrderDbContext.cs
Data/SeedData.cs
Models/CheckoutInput.cs
Models/DemoOrderProgress.cs
Models/Drink.cs
Models/DrinkDraft.cs
Models/OrderLine.cs
Models/PurchaseOrder.cs
Models/ShopSettings.cs
Program.cs
Services/CatalogService.cs
Services/OrderState.cs
Services/PurchaseService.cs
wwwroot/app.css
wwwroot/js/orderTools.js
wwwroot/orders.css
```

Підсумок і запитання для захисту

JS-модуль виконує браузерні дії; `IJSRuntime` передає виклик; `IJSObjectReference` представляє модуль; `DotNetObjectReference` дозволяє JS звернутися до конкретного компонента. Вони мають різні призначення й обидва потребують звільнення.

`OnInitialized` запускає незалежну від DOM роботу; `OnParametersSetAsync` реагує на параметри; `OnAfterRenderAsync` придатний для першої взаємодії з браузером; `DisposeAsync` завершує довготривалу роботу. Це не просто перелік методів: кожен відповідає конкретному моменту життя компонента.

Нові файли: `wwwroot/js/orderTools.js`, `Shared/OrderTools.razor`, `Shared/LiveOrderStatus.razor`, `Models/DemoOrderProgress.cs`; додатково `Shared/PagedList.razor`. Змінені: `CartSummary`, `OrderDetails`, `orders.css`; для додаткового рівня — `Orders`. Сервіс збереження та схема БД не змінюються.

Запитання: чому JS не викликають під час prerendering? Чому таймеру потрібні скасування й `InvokeAsync`? Чому право на Clipboard API не гарантоване? Чим `RenderFragment<T>` відрізняється від готового HTML-рядка?

Запуск і автоматичні перевірки

Із кореня готового комплекту:

```
dotnet run --project CampusCafe --urls http://localhost:5127
```

Для власного проєкту з його папки достатньо `dotnet run --urls http://localhost:5127`. Не запускайте кілька етапів на одному порту. Зупинка — `Ctrl+C`.

Окремо запустіть перевірки демо:

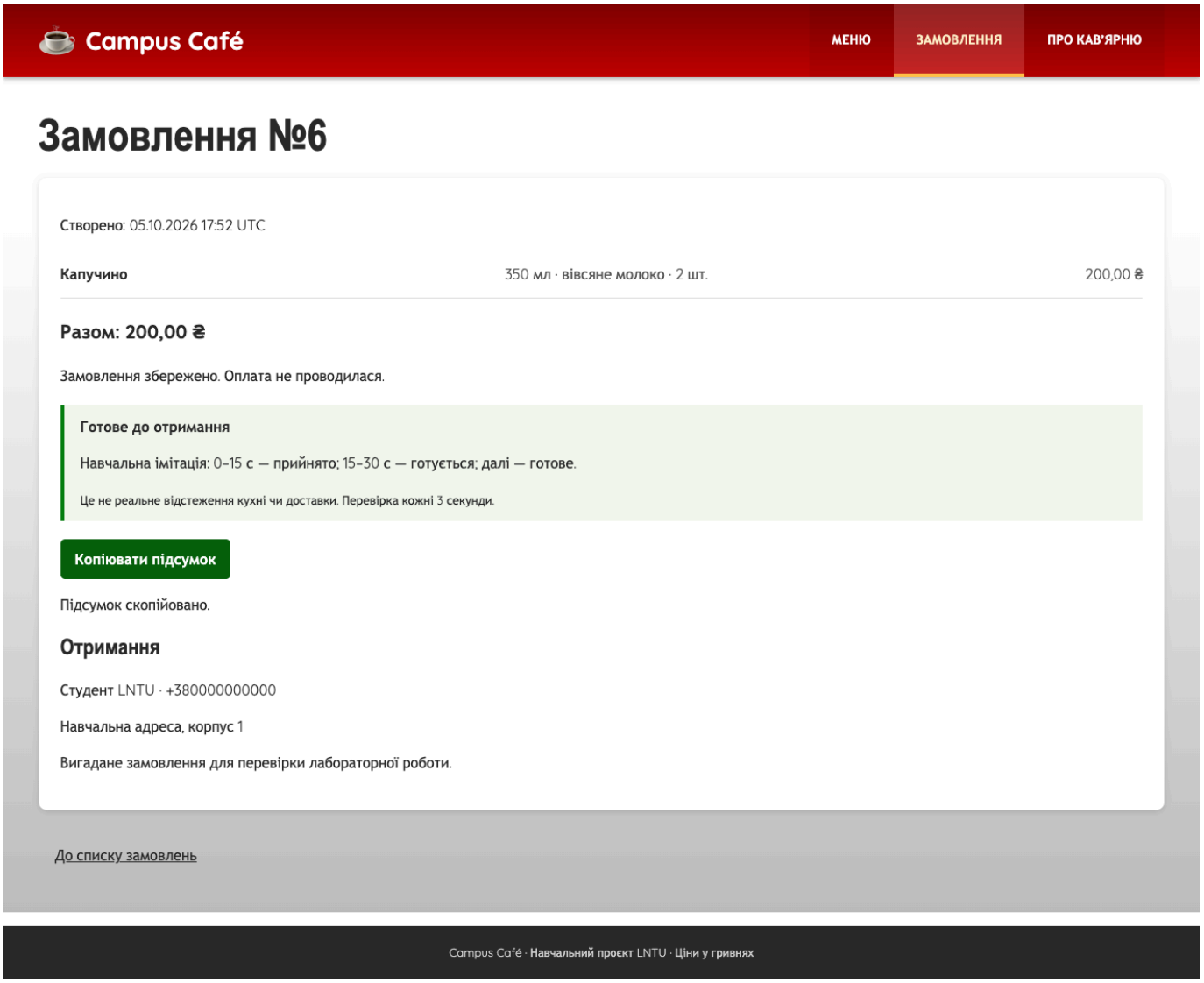
```
dotnet run --project CampusCafe.Checks
```

Тести працюють з окремими SQLite-базами в пам'яті й не змінюють робочі файли. Це перевірки C# і сховища, а не заміна ручного тестування браузера. Проєкт перевірок можна взяти з комплекту поточного етапу; для власних моделей адаптуйте типи, назви й контрольні ціни. Не запускайте старі тести зі старими сигнатурами сервісів.

Для здавання підготуйте вихідний код без `bin`, `obj` і персональних даних, короткий README, результати контрольних сценаріїв та пояснення власної тематичної адаптації. Збережіть окремий працездатний знімок цього етапу.

Джерела для поглиблення

- JavaScript у модулях.
- Виклики .NET із JavaScript.
- Життєвий цикл.
- Шаблонні компоненти.



Короткий підсумок роботи

Що реалізовано. До застосунку додано браузерне підтвердження видалення позиції, копіювання підсумку з ручним запасним варіантом і навчальне оновлення статусу замовлення. Компоненти звільняють посилання та завершують фонову роботу після виходу зі сторінки. Додаткова вправа розширює список універсальною пагінацією.

Як це працює:

- **Виклики C# → JavaScript.** `IJSRuntime` імпортує `orderTools.js`. Через `IJSObjectReference` компонент викликає функції діалогу й буфера обміну. Позиція видаляється лише після підтвердження, а відмова Clipboard API відкриває ручне копіювання.
- **Зворотний виклик.** `DotNetObjectReference` передає JavaScript посилання на конкретний .NET-компонент. Після копіювання JS викликає його метод `OnCopied`, позначений `[JSInvokable]`, щоб показати повідомлення.
- **Життєвий цикл.** Модуль для копіювання завантажується після першого інтерактивного рендерингу. `DisposeAsync` звільняє JS- і .NET-посилання, а перевірка `disposed` захищає від запізнілого оновлення завершеного компонента.

- **Навчальний статус.** `DemoOrderProgress` обчислює етап за різницею UTC-часу, а `PeriodicTimer` періодично запускає перерахунок. `InvokeAsync` і `StateHasChanged` забезпечують оновлення інтерфейсу. Токен скасування та очікування задачі дозволяють коректно зупинити цикл.
- **Додаткова пагінація.** `PagedList<TItem>` вибирає частину вже завантаженого списку через `Skip` і `Take`. Батьківська сторінка задає вигляд елемента через `RenderFragment<TItem>`, тому компонент не залежить від тематики товарів чи замовлень.

Головне запам'ятати: серверний C# і браузерний JavaScript працюють у різних середовищах, тому їх взаємодія асинхронна й може завершитися відмовою. Фонові операції та службові посилання потрібно звільняти. Статус за таймером залишається імітацією, а не підтвердженням реального виконання замовлення.

Тестові питання для самоперевірки

У кожному питанні оберіть **одну правильну відповідь**. Питання 11–12 стосуються додаткової вправи з пагінацією. Спочатку запишіть відповіді, потім порівняйте їх із ключами нижче.

1. Де виконується `navigator.clipboard.writeText(...)` у цьому Interactive Server застосунку?

- А. Усередині SQLite.
- Б. У C#-контексті сервісу замовлень.
- В. У процесі компіляції Razor.
- Г. У браузері користувача, у JavaScript-модулі.

2. Користувач натиснув «Скасувати» в діалозі видалення. Що має зробити C#-код?

- А. Видалити позицію, але залишити стару суму на екрані.
- Б. Не викликати `Order.Remove`, оскільки функція повернула `false`.
- В. Видалити все замовлення з SQLite.
- Г. Повторювати діалог, доки користувач не підтвердить.

3. Чим відрізняються `IJSObjectReference` та `DotNetObjectReference<OrderTools>`?

- А. Перше представляє JS-об'єкт для викликів із C#, друге дозволяє JS звернутися до конкретного .NET-екземпляра.
- Б. Це взаємозамінні назви посилання на один HTML-елемент.
- В. Перше містить замовлення з БД, друге — CSS-стилі.
- Г. Обидва потрібні лише для синхронних викликів без `await`.

4. Чому після першого імпорту модуля в `OnAfterRenderAsync` викликається `StateHasChanged()`?

- А. Щоб повторно завантажити всі товари з БД.
- Б. Щоб скасувати браузерні обмеження буфера обміну.
- В. Щоб інтерфейс урахував готовність модуля й розблокував кнопку.
- Г. Щоб запустити другий незалежний circuit.

5. Браузер заборонив запис до буфера обміну. Яка поведінка правильна?

- А. Показати успіх, адже користувач натиснув кнопку.
- Б. Приховати весь вміст замовлення без пояснення.
- В. Нескінченно повторювати запит доступу.
- Г. Повідомити про відмову та надати текст для ручного копіювання.

6. Що дозволяє `receiver.invokeMethodAsync("OnCopied")` звернутися до потрібного компонента?

- А. Збіг назви файла JavaScript із назвою таблиці БД.
- Б. Передане посилання `DotNetObjectReference` та доступний метод із `[JSInvokable]`.
- В. Будь-який публічний рядок у `App.razor`.
- Г. Атрибут CSS-класу `order-tools`.

7. Чому `DemoOrderProgress.GetStatus` отримує `nowUtc` параметром?

- А. Це дозволяє перевіряти різні часові межі без очікування справжнього таймера.
- Б. Так функція автоматично отримує статус із кухні.
- В. Це робить SQLite-запит синхронним.
- Г. Так статус зберігається в браузерному буфері обміну.

8. Навіщо таймер передає оновлення через `InvokeAsync` і викликає `StateHasChanged`?

- А. Щоб відкрити нову сторінку без URL.
- Б. Щоб записати кожний сигнал таймера в БД.
- В. Щоб виконати зміну в контексті рендерера й запросити оновлення інтерфейсу.
- Г. Щоб автоматично прискорити виконання таймера вдвічі.

9. Яка правильна послідовність завершення фонового циклу в `DisposeAsync`?

- А. Звільнити джерело токена й залишити цикл працювати.
- Б. Лише приховати текст статусу через CSS.
- В. Запустити ще один таймер для контролю старого.
- Г. Подати сигнал скасування, дочекатися завершення задачі та звільнити ресурси.

10. Чому перехід до старого замовлення не запускає його навчальний статус із початку?

- А. Браузер назавжди запам’ятовує останній текст кожного компонента.
- Б. Формула використовує збережений `CreatedUtc`, а не час відкриття сторінки.
- В. `OnInitialized` ніколи не викликається після першого запуску сайту.
- Г. `@key` автоматично записує статус у SQLite.

11. Що передає `RenderFragment<TItem> ItemTemplate` компоненту `PagedList`?

- А. Шаблон відображення, який отримує один елемент типу `TItem`.
- Б. Готовий файл бази даних.
- В. Лише кількість сторінок.
- Г. Рядок JavaScript для зміни URL.

12. Є 7 записів, `PageSize = 3`, а внутрішній індекс `page = 1`. Що покаже компонент?

- А. Перші три записи, напис «Сторінка 1 із 2».
- Б. Лише сьомий запис, напис «Сторінка 3 із 3».
- В. Четвертий, п’ятий і шостий записи, напис «Сторінка 2 із 3».
- Г. Усі сім записів, бо пагінація працює лише в БД.

Ключі відповідей і пояснення

№	Відповідь	Чому
1	Г	Clipboard API належить браузеру. Серверний C# звертається до нього через JavaScript interop.
2	Б	Зміна кошика виконується лише після позитивної відповіді. Відмова не повинна мати побічного ефекту видалення.
3	А	Посилання забезпечують різні напрямки взаємодії. Кожне має власне призначення і потребує звільнення.
4	В	Завершення `OnAfterRenderAsync` саме по собі не запускає наступний рендеринг, а стан кнопки залежить від `module`.
5	Г	Наявність API не гарантує дозволу. Запасний ручний варіант дозволяє продовжити роботу без хибного повідомлення успіху.
6	Б	JS отримує посилання на конкретний об'єкт, а `[JSInvokable]` відкриває відповідний метод для виклику.
7	А	Тест може підставити моменти до та після межі. Це перевіряє формулу незалежно від роботи таймера.
8	В	Таймер не є звичайною DOM-подією. Оновлення потрібно передати рендереру та повідомити про зміну стану.
9	Г	Скасування завершує очікування, `await loop` дочікується виходу, після чого ресурси можна коректно звільнити.

№	Відповідь	Чому
10	Б	Вік замовлення обчислюється від часу його створення. Статус є імітацією й не підтверджує реального виконання.
11	А	Компонент пагінації вибирає елементи, а батьківська сторінка визначає вигляд кожного з них через шаблон.
12	В	`Skip(1 * 3).Take(3)` пропускає перші три записи. Для семи записів потрібно три сторінки, а на екрані індекс збільшується на один.